

Debacu Revenue Intelligence

Documento base de Fase 1

Tablas, relaciones y políticas

Base de referencia para continuar la implementación técnica de Revenue Intelligence *sin duplicar Stripe, planes, organizaciones, members ni la seguridad actual de Debacu.*

Principio rector: Revenue Intelligence se apoya en el core actual de Debacu. No crea un sistema paralelo de suscripción, organizaciones o acceso; añade únicamente las tablas y procesos necesarios para propiedades, configuración operativa, datos diarios, pickup e importación CSV.

1. Objetivo del documento

Este documento fija la base funcional y técnica de la Fase 1 de Revenue Intelligence en Debacu. Su objetivo es que el siguiente trabajo —SQL, políticas RLS, servicios, Edge Functions e importación CSV— siga una misma lógica y no se desvíe del modelo que ya funciona en Debacu Evaluation.

- Mantener intacto el core actual de Debacu: suscripciones, Stripe, organizaciones, miembros, JWT y control por plan.
- Añadir el módulo de Revenue Intelligence de forma progresiva, sin reemplazos bruscos ni duplicidades innecesarias.
- Empezar por la capa mínima útil: propiedades, tipos de habitación, temporadas, eventos, revenue diario, pickup e importaciones.
- Dejar fuera en esta fase todo lo que complique sin aportar valor inmediato: RMS avanzado, forecast complejo, pricing engine o modelo completo de reservas granulares.

2. Principios de diseño

- No duplicar entidades existentes. Revenue usa el modelo actual de Debacu para organización, membresía, permisos y planes.
- Modelo incremental. Se habilita una Fase 1 productiva mientras el resto de Debacu sigue estable y listo para producción con hoteles piloto.
- Mismo patrón de seguridad. Todo debe seguir la filosofía actual: JWT, org_id, membresías y RLS por organización.
- CSV primero, PMS/API después. La ingesta inicial se apoya en importaciones robustas, trazables y repetibles.

- Métricas fiables antes que complejidad. Se prioriza revenue diario y snapshots de pickup antes que un modelo exhaustivo de reservas.

3. Alcance funcional de Fase 1

La Fase 1 debe permitir construir y alimentar las primeras pantallas del módulo Revenue Intelligence:

- Propiedades
- Dashboard
- Día a día
- Pickup avanzado
- Canales & Segmentos
- Importación CSV

Informes podrá apoyarse en esta base, pero no necesita un motor completo adicional en esta fase. La pantalla Mi cuenta no forma parte del módulo de Revenue porque ya existe en Debacu y está conectada a Stripe y al control de planes.

4. Modelo de datos base de Fase 1

Las tablas de Fase 1 se dividen en tres bloques: configuración de propiedades, datos operativos de revenue y pipeline de importación.

Tabla	Bloque	Propósito operativo	Estado esperado en Fase 1
debacu_eval_properties	Configuración	Catálogo de propiedades por organización.	Obligatoria
debacu_eval_property_room_types	Configuración	Tipos de habitación por propiedad.	Obligatoria
debacu_eval_property_seasons	Configuración	Temporadas por propiedad.	Obligatoria
debacu_eval_revenue_events	Configuración	Eventos con impacto en demanda o precio.	Obligatoria
debacu_eval_revenue_daily	Datos	Dato diario consolidado para dashboard, día a día y canales/segmentos.	Obligatoria
debacu_eval_revenue_pickup_snapshots	Datos	Fotos históricas para cálculo de pickup.	Obligatoria

debacu_eval_revenue_imports	Ingesta	Cabecera de importación CSV con estado y trazabilidad.	Obligatoria
debacu_eval_revenue_import_errors	Ingesta	Detalle de errores por fila/campo en importación.	Obligatoria

4.1 Propiedades

debacu_eval_properties guarda la relación entre organización y propiedad operativa. No sustituye al modelo central de organizaciones; lo extiende para Revenue.

- Relación principal: org_id → debacu_eval_organizations(id).
- Campos funcionales mínimos: code, name, city, country, timezone, rooms_total, is_active.
- Campos auxiliares útiles: legal_name, tax_id, property_type, region, currency_code, created_by.
- Regla de negocio: el número de propiedades activas se controlará por plan, pero esa validación se hará en servicio/Edge Function, no duplicando lógica de planes en SQL.

4.2 Tipos de habitación

debacu_eval_property_room_types permite estructurar la oferta de cada propiedad y preparar después lógica de pricing, segmentación y análisis por inventario.

- Relaciones: org_id → organizations; property_id → properties.
- Campos clave: code, name, capacity, rooms_count, base_price, is_active.
- Restricción esperada: unique(property_id, code).
- Uso posterior: filtros de dashboard, revenue diario por tipo de habitación y formularios de configuración.

4.3 Temporadas

debacu_eval_property_seasons modela bloques temporales operativos por propiedad.

- Relaciones: org_id y property_id.
- Campos clave: season_type, start_date, end_date, priority, is_active.
- Regla mínima: end_date >= start_date.
- Uso posterior: vista comercial, reglas futuras de pricing y análisis contextual del revenue.

4.4 Eventos

debacu_eval_revenue_events registra contexto externo o interno con impacto potencial en demanda y precio.

- Relaciones: org_id y property_id.
- Campos clave: name, event_type, impact_level, start_date, end_date, note, is_active.
- Uso posterior: cruce entre ocupación/pickup y eventos relevantes.

4.5 Revenue diario

debacu_eval_revenue_daily es la tabla central del módulo en Fase 1.

- Relaciones: org_id, property_id y room_type_id opcional.
- Grano esperado: property + stay_date + channel + segment + room_type.
- Campos clave: rooms_sold, rooms_available, revenue_rooms, revenue_total, adr, revpar, source.
- Pantallas que alimenta: Dashboard, Día a día y Canales & Segmentos.

4.6 Pickup snapshots

debacu_eval_revenue_pickup_snapshots conserva fotografías históricas del on-the-books para poder calcular pickup real.

- Relaciones: org_id y property_id.
- Campos clave: snapshot_date, stay_date, channel, segment, rooms_sold, revenue_rooms.
- Grano esperado: property + snapshot_date + stay_date + channel + segment.
- Pantalla que alimenta: Pickup avanzado.

4.7 Importaciones

debacu_eval_revenue_imports y debacu_eval_revenue_import_errors dan trazabilidad a la entrada de datos.

- Import cabecera: tipo, estado, ruta, hash, contadores, resumen de errores y metadatos.
- Import errores: fila, campo, código y mensaje.
- Patrón recomendado: validar primero, registrar errores, y solo después hacer commit de los datos buenos.

5. Relaciones y dependencia con el core actual de Debacu

Revenue Intelligence no define su propio sistema de acceso ni su propio catálogo de planes. La dependencia correcta con el core actual es la siguiente:

Entidad base ya existente	Uso desde Revenue
debacu_eval_organizations	Entidad raíz de tenant; todas las tablas de Revenue cuelgan de org_id.
debacu_eval_org_members	Base de autorización y RLS por membresía/rol.
JWT / auth.uid()	Identificación del usuario autenticado.
Sistema actual de suscripción / Stripe	Control de plan, límites y entitlements. Revenue lo consulta; no lo duplica.

La validación de límites —por ejemplo, cuántas propiedades puede crear cada organización según plan— debe ejecutarse en servicios o Edge Functions con service role, leyendo el plan real de Debacu. No debe materializarse un segundo catálogo de planes dentro de Revenue.

6. Política de seguridad y RLS

La seguridad de Fase 1 debe seguir exactamente el patrón del resto de Debacu: aislamiento por organización, control por membresía y escritura restringida por rol.

Capa	Regla base
Frontend	Mostrar/ocultar pantallas según plan y permisos, pero sin confiar solo en la UI.
Ruta	No permitir acceso por URL directa a una opción no habilitada.
Base de datos	RLS habilitado en todas las tablas de Revenue.
Policies de lectura	Un usuario autenticado solo ve registros de organizaciones donde es miembro.
Policies de escritura	Solo OWNER y ADMIN pueden crear/editar/borrar configuración o datos importados.
Edge Functions	Usar service role solo cuando haga falta validar límites, importar CSV o hacer procesos internos.

Helpers recomendados de RLS: una función para comprobar acceso a una organización y otra para comprobar si el usuario es OWNER o ADMIN en esa organización. Todas las policies de Revenue deben colgar de ese patrón.

7. Orden de implementación recomendado

Orden	Pieza	Motivo
1	Cerrar SQL base y RLS	Evita seguir construyendo sobre tablas incompletas o sin seguridad.
2	Pantalla Propiedades	Es la base del módulo y el punto de entrada de configuración.
3	Edge Function de gestión de propiedades	Debe validar org, rol y límite por plan.

4	Importación CSV Revenue Daily	Es la forma más realista de empezar a alimentar datos.
5	Importación CSV Pickup	Permite activar Pickup Avanzado sin depender de PMS integrado.
6	Dashboard	Da valor visual rápido con datos ya consolidados.
7	Día a día	Consulta directa sobre revenue_daily.
8	Canales & Segmentos	Agregación sobre revenue_daily.
9	Pickup avanzado	Comparativa de snapshots.

8. Qué queda fuera de Fase 1

- Duplicar My Account, planes o gestión Stripe dentro de Revenue Intelligence.
- Crear un segundo modelo de organizaciones o miembros.
- Meter un RMS completo con pricing automático, overbooking, optimización avanzada o forecast complejo.
- Forzar desde el inicio una integración PMS universal. La prioridad es CSV robusto y trazable.
- Basar el módulo en reservas granulares si eso retrasa la entrega. En Fase 1 manda el dato útil y consistente.

9. Checklist de cierre de Fase 1 base

Ítem	Criterio de cierre
Tablas	Las ocho tablas de Fase 1 existen y tienen sus columnas clave.
Relaciones	Todas las FK principales contra organizations, properties y room_types están creadas.
RLS	RLS activo en todas las tablas de Revenue.
Policies	Policies SELECT y WRITE creadas y probadas.
Importación	Cabecera y errores de importación operativos.
Frontend	Pantalla Propiedades en funcionamiento.

Datos	Al menos una propiedad con room types, temporadas, eventos y una importación válida de revenue/pickup.
-------	--

10. Decisión base para continuar

La decisión técnica correcta para DebaCu es continuar Revenue Intelligence como módulo nuevo, apoyado en el core existente y alimentado en Fase 1 por propiedades, configuración operativa, revenue diario, pickup e importación CSV. Este documento sirve como referencia para que los siguientes SQL, servicios y Edge Functions mantengan coherencia y no introduzcan duplicidades ni desviaciones arquitectónicas.